

Bona IT Helpdesk Backend API Documentation

1. Project Overview

The Bona IT Helpdesk Backend API is a RESTful service built using **Next.js 16 (App Router)** and **TypeScript**.

It provides the core functionality required for a simple IT support ticketing system where users can submit issues and agents can manage them.

This backend is designed using **in-memory storage**, meaning all data is temporarily stored during runtime for development and testing purposes.

2. Technologies Used

- Next.js 16 (App Router)
 - TypeScript
 - Node.js Runtime
 - In-memory JavaScript Arrays (temporary storage)
 - REST API Architecture
-

3. System Features

3.1 Ticket Creation

Users can submit IT support requests.

Example Request:

```
{  
  "name": "Sebastian",  
  "email": "sebastian@example.com",  
  "issue": "Printer not working"  
}
```

Example Response:

```
{  
  "success": true,
```

```
"message": "Ticket created successfully",  
"data": {  
  "id": "1781700302355",  
  "token": "48GRYL",  
  "status": "Open"  
}  
}
```

3.2 Get All Tickets

Retrieves all tickets for the admin/agent dashboard.

Endpoint:

GET /api/tickets

Example Response:

```
{  
  "success": true,  
  "data": [  
    {  
      "id": "1781700302355",  
      "name": "Sebastian",  
      "status": "Open"  
    }  
  ]  
}
```

3.3 Get Single Ticket

Fetches a specific ticket using its unique ID.

Endpoint:

GET /api/tickets/[id]

Example:

/api/tickets/1781700302355

3.4 Ticket Status Lookup (Client Access)

Allows clients to check ticket status without logging in using a unique token.

Endpoint:

GET /api/tickets/status/[token]

Example:

/api/tickets/status/48GRYL

Response:

```
{  
  "success": true,  
  "ticketId": "1781700302355",  
  "status": "Resolved"  
}
```

3.5 Assign Ticket to Agent

Allows an agent to assign a ticket to themselves or another staff member.

Endpoint:

PUT /api/tickets/[id]/assign

Example Request:

```
{  
  "assignedTo": "Miles"  
}
```

3.6 Update Ticket Status

Allows updating ticket progress.

Endpoint:

PUT /api/tickets/[id]/status

Example Request:

```
{  
  "status": "Resolved"  
}
```

Status Options:

- Open
- In Progress
- Resolved

4. Data Model

Each ticket follows this structure:

```
type Ticket = {  
  id: string;  
  token: string;  
  name: string;  
  email: string;  
  issue: string;  
  status: "Open" | "In Progress" | "Resolved";  
  assignedTo: string | null;  
  createdAt: string;  
};
```

5. Architecture Design

The system uses a simple REST API structure:

Client Request → Next.js API Route → In-Memory Storage → JSON Response

Example Flow:

1. User submits ticket

2. API generates ticket object
 3. Ticket stored in memory array
 4. Response returned in JSON format
-

6. Limitations

- Data is not persistent (resets on server restart)
 - No database integration (future improvement with Prisma/PostgreSQL)
 - No authentication system implemented (agents are not secured)
-

7. Future Improvements

- Integrate PostgreSQL with Prisma ORM
 - Add NextAuth.js authentication for agents
 - Email notifications using Nodemailer
 - Agent dashboard UI
 - SLA tracking system
-

8. Conclusion

The Bona IT Helpdesk Backend API successfully implements a functional ticket management system using modern web development practices. It demonstrates RESTful API design, dynamic routing, and in-memory data handling using Next.js 16.

This backend serves as a strong foundation for future expansion into a full production-ready helpdesk system.
